12. **Fidelity Considerations:** For each component/service of the DT, how they correspond to the SUS. For example, whether a reduced-order model was built to balance fidelity versus performance [29].
13. **Life-cycle Stages:** The stages of the SUS (*ideation*, *realisation*, *utilisation*, etc.) for which the DT is utilised. If the scope of the SUS changes, this should also be reported.
14. **Evolution:** The evolution of the DT throughout its development.

Note that this DT description framework has been recently extended to 21 characteristics in [12] by merging three DT frameworks, including the one above, to present a unified characterization of DTs. [12] adapts the material found here to provide a detailed description for all 21 characteristics for the Flex-cell use case. As well, the Incubator and Robotti DTs found in this chapter are also summarized in tabular form.

12.3 The Tempeh Incubator

The tempeh incubator example introduced in Sect. 3.2 has been used through this book as a representative example of a system to be controlled by a DT. The business context involving the Twinpeh company has been slightly fictionalised, but a real incubator and DT have been built at AU. Here, we return to the real incubator DT to provide an introductory example showing the usage of the DT description framework. We summarise the incubator case here, and also provide links to those sections of the book which contain additional context for each characteristic of the incubator.

12.3.1 Incubator DT Overview

The purpose of the PT is to maintain a specific temperature within the incubator's chamber to support the fermentation of tempeh. The primary purpose of the incubator's DT is to enhance the functionality of the PT through the DT services. As mentioned in Sect. 3.5, these services support business goals including, but not limited to:

1. Visualising the incubation process.
2. Conserving energy, for example, by turning off the controller when the lid is opened.
3. Signalling when a batch of tempeh is ready.
4. Indicating when equipment requires maintenance.
5. Detecting subpar batches of tempeh.

6. Measuring and suggesting control actions to improve the quality of tempeh.

For the co-simulation of the flex-cell we use Maestro and the RabbitMQFMU [7]. A graphical representation of the incubator PT and its DT in Fig. 12.1. In this figure, the PT is on the left, while its DT is on the right. The following sections will refer to this figure to explain the conceptual components and characteristics of the incubator and its DT.

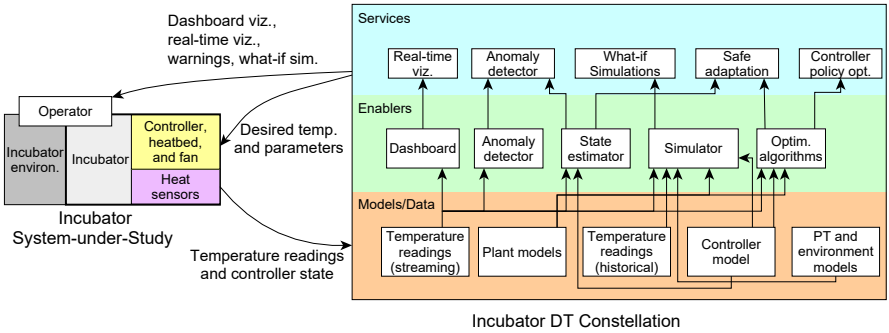

Fig. 12.1: The incubator system and its Digital Twin.

12.3.2 Fundamental Characteristics

This section presents the incubator case study through the 14 fundamental DT characteristics as introduced in Sect. 12.2.

12.3.2.1 C1: System-under-study

The SUS includes the incubator PT along with its environment. The elements of the SUS are described in Table 12.1, organised hierarchically.

The main interactions between the incubator and its environment are:

- the room is assumed to be colder than the incubator. It therefore acts as a heat sink, contributing to cooling down the incubator's box and the air inside the incubator.
- the user can open the lid to check on the tempeh incubator. When the lid is open, the cooling-down process is accelerated.

Table 12.1: Components of the incubator SUS.

	Part	Description
Plant	Power supply	Powers electrical components
	Fan	Circulates air throughout box
	Heater	Raises the temperature of surrounding air
	Electrical switching board	Allow controller to turn on/off heater/fan
	Temperature sensors	Two inside box, one outside
	Styrofoam™ box	With lid, to contain components and air
	Air volume	
	Tempeh	Object to be temperature-controlled
Controller	Running on Raspberry PI	Controls the fan and heater
Environment	Surface	Where the incubator is placed
	Surrounding air	Heat sink
Agents	User	Manual operation of the lid

12.3.2.2 C2: Acting Components

Two components were added and/or updated to allow the DT to interact with the SUS. First, the controller of the incubator was updated to connect to a secure RabbitMQ server (see Sect. 7.4), for checking whether there are requests to update the controller parameters and desired temperature. Second, a safety layer was coded into the controller that discards non-essential values for parameters and/or desired temperatures. This mitigates errors in the DT from propagating to the incubator and creating an unsafe situation, such as raising the temperature too high.

12.3.2.3 C3: Sensing Components

To allow the PT to communicate directly with the DT, the controller was modified to add two abilities. The first ability is to connect to a secure RabbitMQ server and upload the controller and incubator state. This includes the heater state, the temperatures measured and all other necessary information as mentioned below. The second ability is to recover when the network connection drops, to improve the robustness of the DT-PT communication.

12.3.2.4 C4: Multiplicities

Currently, there is only one DT connected to one incubator. In the future, it is anticipated that multiple incubators could be controlled by one DT. Note that in this scenario, the heat produced by one incubator may affect another one. This would require a more sophisticated composition approach for analysis and control over a fleet of incubators (see Sect. 11.11).

12.3.2.5 C5: Data Transmitted

The data sent periodically to the DT by the controller is comprised of: a) the temperature of each sensor, b) the average temperature used by the controller to make a decision, c) the time it takes for each control cycle, d) the current state of the controller (e.g., waiting or cooling down, as in Fig. 12.2), and e) the values of each of the controller parameters.

12.3.2.6 C6: Insights/Actions

As one *insight*, the incubator DT provides a dashboard for performing monitoring of the incubator's state, such that the operator understands the incubator's behaviour at a glance. This dashboard serves as a sink for all things that can be visualised related from the different DT services. For instance, the Lid Opening Service will produce data that is visualised in the dashboard (see Sect. 8.4). Other insights include *real-time visualisation* of the incubator, *warnings* about anomalous behaviour, and *what-if simulation* results for the user.

The automatic *actions* that the DT performs on the PT depend on the controller version being used. There are different versions of the controller, but generally the actions send the parameters to be set on the controller, as well as the desired temperature. The simplest controller version, with a state machine model as introduced in Sect. 5.6.1, can have its lower and upper bounds of temperature set by the DT. It can also be commanded to shutdown. A more advanced version of the controller contains waiting states that allow for finer tuning of its behaviour in the presence of more powerful heating elements. This version is illustrated in Fig. 12.2. In this advanced version the DT can configure, in addition to the lower and upper bounds, the parameters C and H in seconds. The above actions can be leveraged to, for example, turn off the controller when the lid is detected to be open.

12.3.2.7 C7: Services

The DT currently provides the following services for the incubator:

- *Real-time visualisation* for the user to understand the incubator's current state at a glance (see Sect. 8.4).
- An *anomaly detector* which alerts the user to any anomalous behaviour that may interfere with proper incubation, such as the lid being open for too long (see Sect. 9.4).
- Providing *what-if simulations* such that the user can enter in proposed conditions and examine the resulting behaviour (see Sect. 10.2).

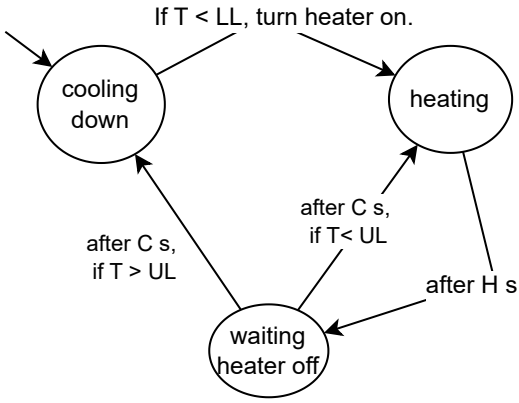

Fig. 12.2: State machine of the controller containing waiting states.

- *Safe adaptation when the lid is opened* to adjust the controller parameters when the lid has been opened for too long (Sect. 10.5). This saves energy, and can prevent unsafe situations where the heater runs at maximum power.
- *Controller policy optimisation when PT changes*. When a component of the system is changed, such as the heater or power supply, the DT will detect that the incubator physical models used are no longer a valid representation of the PT configuration. A re-calibration of such models will be triggered, as well as re-optimisation of the control parameters (see Sect. 10.5).
  - One adaptation sub-process will re-calibrate the models of the PT if the dynamics have been detected to change.
  - A second adaptation sub-process will then re-compute the optimal control policy if the dynamics have changed.

12.3.2.8 C8: Enablers

There are several enablers that support the incubator DT services:

- A *simulator*, which is able to simulate the behaviour of the incubator forward in time and perform what-if experiments.
  - There are two options: a) the *incubator* itself can be simulated for self-adaption purposes, or b) the combination of the *controller and the incubator* can be simulated for control policy optimisation.
- An *anomaly detector* that can determine what is the normal behaviour of the incubator, what is anomalous behaviour, and can alert the user to anomalies.

- A *state estimator* for the incubator which predicts the current state (open/closed lid).
- *Optimisation algorithms* that are used for calibration purposes and the optimisation of the controller behaviour.
- The real-time *dashboard* for displaying information/visualisations.

12.3.2.9 C9: Models and Data

Before detailing the models, we summarise the main quantities of importance about the system, as well as their symbolic representation and units:

- T_b – Average temperature of the air volume inside the box (°C).
- T_h – Temperature of the heating element (°C).
- T_r – Room temperature (°C).
- H_{Heater} – State of the heater (on, presented as 1, and off represented as 0)
- UL – Controller temperature upper limit (°C)
- LL – Controller temperature lower limit (°C)
- C – Cooldown timer (s)
- H – Heating timer (s)

The incubator has the following models for the incubator, which we organise according to their focus.

Plant Models

2 Parameter Model: A simple scalar Ordinary Differential Equation (ODE) representing the plant dynamics. It takes as inputs T_r, H_{Heater}, and returns as an output T_b. The parameters include the heat transfer coefficient between the air volume inside the box and the room, and the heat capacity of the air volume. The initial condition of this model is the initial temperature of the air volume $T_b(0)$.

4 Parameter Model: A plant dynamic model (also an ODE), introduced in Sect. 5.4.1, that takes as inputs T_r, H_{Heater}, and returns as an output T_b and T_h. The parameters include the same parameters as the 2 parameter model plus the heat transfer coefficient between the heater and the air volume as well as the heater capacitance. The initial conditions are: The initial temperature of the heater as well as the air volume.

ANN Model: An Artificial Neural Network (ANN) based model, that takes as inputs the room temperature, the state of the heater, the previous average temperature of the air volume inside the box, and returns as outputs the average temperature of the air volume inside the box.

ANN State Estimation Model: An ANN based model, that takes as inputs the room temperature, the state of the heater, the previous average temperature of the

air volume inside the box, and returns as outputs the average temperature of the air volume inside the box as well as the temperature of the heating element.

Controller Model

A controller model (State machine), introduced in Sect. 5.6, that takes as inputs the average temperature of the air volume T_b and outputs the state of the heater T_h. The parameters include the temperature upper and lower limits. The initial condition represents the initial state of the controller (typically cooling down).

Physical Twin Model

The hybrid automata model introduced in Sect. 5.7.2. The input is the room temperature, and the set of parameters is the union of the 4 Parameter model and the controller state machine model.

Environment Model

A simple algebraic model $T_r = A\sin(Bt) + C$ representing the temperature of the room as a function of the time (in seconds). The parameters are A, B, C.

PT + Environment Model

The model introduced in Sect. 5.7. Regarding the data, we first look at the historical data (versus the streaming data from the PT [31]), exemplifying one of the datasets of the incubator, and subsequently we look at the data used to configure the DT services.

An example dataset can be found online[1], which includes a description of the goal of the experiment conducted, the author, the configuration of the system, a log of the experiment, the CSV files containing the raw data, and plots of the results. The CSV files contain all the data sent from the PT to the DT.

The data used to configure the DT services is stored in configuration files, that are loaded when the services are launched. Such configuration file contains the following sections:

RabbitMQ: Contains the configuration parameters of the access to the routing queue server, for instance, the IP address, port, username, password, etc.

InfluxDB: Contains the configuration for the access to the time series database including the url, the access token, and so on.

[1] https://digital-twin-book.org/

Physical Twin: Contains the initial parameters of the controller.
Digital Twin: This is further subdivided into different sections containing service specific configurations, and the different sets of parameters for each model.

Interfaces to Models

In this case study two interfaces are been used to interact with the models: the Functional Mockup Interface (FMI) has been used for co-simulation studies involving the above models (recall Sect. 5.7.1); as well as an FMI-like interface implemented in the python programming language. Examples of models implementing each interface can be found online.[2][3]

12.3.2.10 C10: Constellation

Fig. 12.1 shows how the models/data, enablers, and services provide information to each other. We can consider the top-level services as being supported by a particular "slice" through the enablers and models/data which can be inferred by the information flow arrows in Fig. 12.1.

12.3.2.11 C11: Time-Scale

The DT for the incubator operates at different rates:

- **Slower-than-real-time:** The anomaly alerts and dashboard information provided to the user may not be acted upon in real-time.
- **Real-time:** Dashboard updates, fan/heat-bed control/adaptation, temperature readings.
- **Faster-than-real-time:** The simulator is able to simulate into the future.
  - This must be performed for both re-calibration, and for controller optimisation.
  - The simulator must be able to run hundreds of simulations within a few seconds to support these services.

[2] https://sites.google.com/view/fm2021tutorialdt/home
[3] https://digital-twin-book.org/

12.3.2.12 C12: Fidelity Considerations

The dashboard does not need to be extremely precise for the user. The temperature propagation model on the other hand, must be very accurate (*high-fidelity*) as the incubator temperature must be simulated into the future. It is also important that the *validity frame* [36] of the incubator models must encompass the control policies that are going to be simulated.

In this section we discuss the different procedures we have followed to quantify the accuracy of each of the models produced above.

Plant Models

2 and 4 Parameter Models These two models were calibrated following the procedures introduced in Sect. 6.4.3: experiments were conducted with an empty incubator, with a fixed control signal that just turns ON the heater for specific periods of time. A plot of the example dataset is available online[4] can be seen in Fig. 12.3, and a plot of the simulation results for each of the models can be seen in Fig. 12.4. As can be seen from the plots, for that particular control profile and room temperature conditions the 4 parameter model is very close to the data, whereas the two parameter model is not so accurate for the prediction of the temperature evolution. Nevertheless this model could be used as a very fast predictor of how much energy is required over a long period of time.

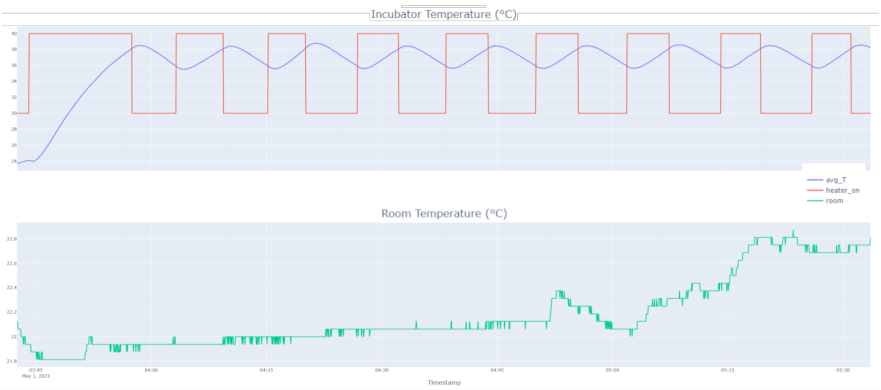

Fig. 12.3: Example dataset used for calibration of the ODE models for the Incubator.

[4] https://digital-twin-book.org/

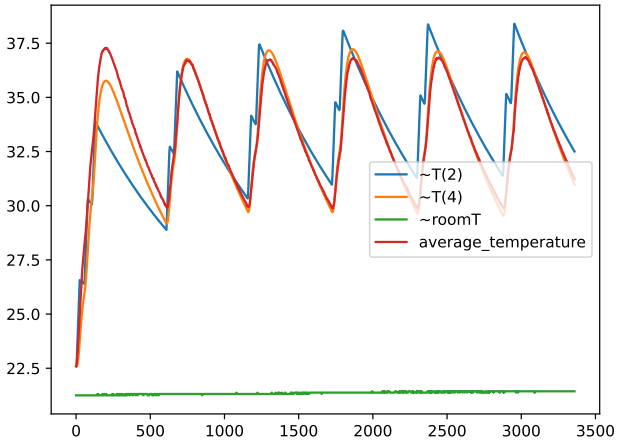

Fig. 12.4: Simulation results for the 2 and 4 parameter Incubator plant models.

ANN Model The calibration procedure of this model was very similar to the calibration procedure of the 2 and 4 parameter models. The model is a simple neural network of a single layer that can take the previous N temperature measurements of the incubator air volume and heating states, and outputs the next temperature estimate (the estimated value of the temperature after 3 seconds, which corresponds to the sample interval in the data set.). It is worth noting that what this model is calibrated to do is to perform one step prediction. This is very different than the use case of the ODE models which are given to a solver and can predict a temperature evolution for many steps. The size of the first layer depends on the parameter N and is given by $2N$. The results of the one step prediction are shown in Fig. 12.5 for $N = 2$. For this model to be used for multistep prediction each temperature predictor needs to be fed back into the model with an iterative evaluation. The errors caused by each prediction quickly accumulate and cause large divergences in multi step predictions, as is illustrated in Fig. 12.6.

ANN State Estimation Model A neural network based model, that takes as inputs the room temperature, the state of the heater, the previous average temperature of the air volume inside the box, and returns as outputs the average temperature of the air volume inside the box as well as the temperature of the heating element.

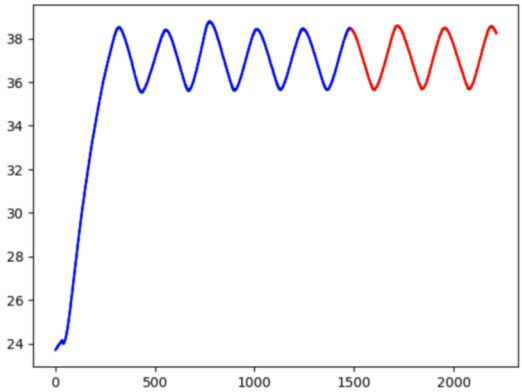

Fig. 12.5: One step prediction results for the Incubator ANN model. The real data set is underneath the blue and red lines. The red line represents data that has not been used to train the neural network demonstrating its ability to generalise.

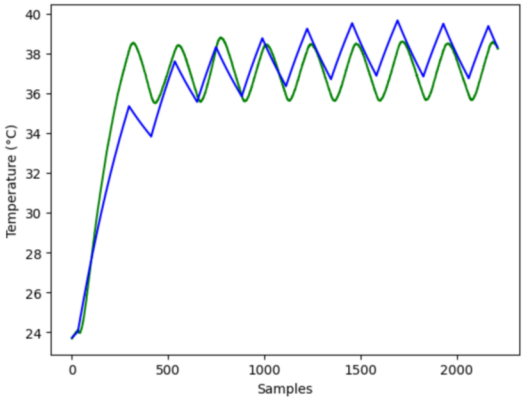

Fig. 12.6: Multi step prediction results for the ANN model (blue line), compared to the data set (green line). This is an example of a bad usage of this Incubator model, because even if each step prediction error is small, those small errors are being fed back into the network and getting amplified.

Controller Model

The parameters of this model can be determined exactly from inspection of the controller code. Its calibration is therefore trivial.

Remaining Models

The remaining models represent compositions of the above models and therefore do not need to be calibrated. However, it is worth discussing their accuracy. As can be seen in the Fig. 12.7, the predicted temperature follows a similar shape as a temperature in the dataset but these two signals become misaligned over time. This divergence occurs because small inaccuracies in the temperature measurements will cause small inaccuracies in the time at which the controller transitions are taken. The difference in the controller transition time in turn leads to the temperature dropping earlier or later than the real temperature in the data set, therefore exacerbating the differences between the two signals. These inaccuracies accumulate overtime leading to total divergence between the two signals. This is one of the main reasons one needs to use the data fusion techniques introduced in Sect. 7.6.

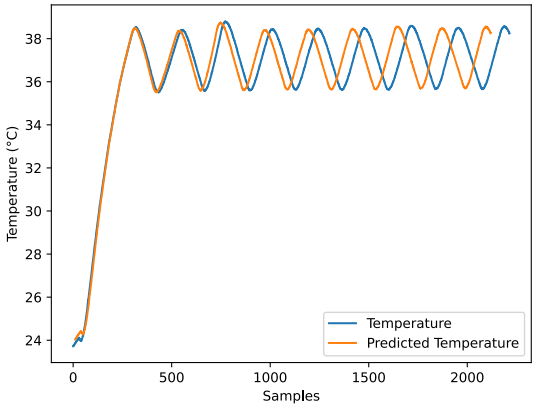

Fig. 12.7: Example co-simulation results and comparison with dataset for the Incubator. The figure shows an example state divergence which is a common phenomenon in models.

12.3.2.13 C13: Life-cycle Stages

All the services provided by this DT are for the *operation* life-cycle stage [35] of the incubator.

12.3.2.14 C14: Evolution

The evolution of the incubator DT is reported in [5, 4]. It can be summarised as follows, and is illustrated in Fig. 12.8.

1. The relevant physics were identified before building the first version of the PT.
2. From this preliminary plant models were built with rough guesses for the parameters.
3. This enabled us to answer questions such as how powerful and heater was necessary for our incubator.
4. The first prototype of the incubator was built.
5. Lab experiments were carried out to refine the estimates for the parameters of the models and compare the different plant models.
6. A controller model was built and the parameters of the controller model were tuned in co-simulation together with a plant model.
7. The controller code was deployed into the physical controller of the incubator and tested.
8. The visualisation service was the first DT service to be deployed.
9. Subsequently the state estimation and normal detection services were implemented.
10. The self adaptation loop was then implemented using the models to optimise the controller whenever the lid was opened.

It is worth highlighting that the preliminary DT services have enabled us to reach new insights which affected the implementation of subsequent DT services, thus highlighting one of the big benefits of DTs and their incremental development.

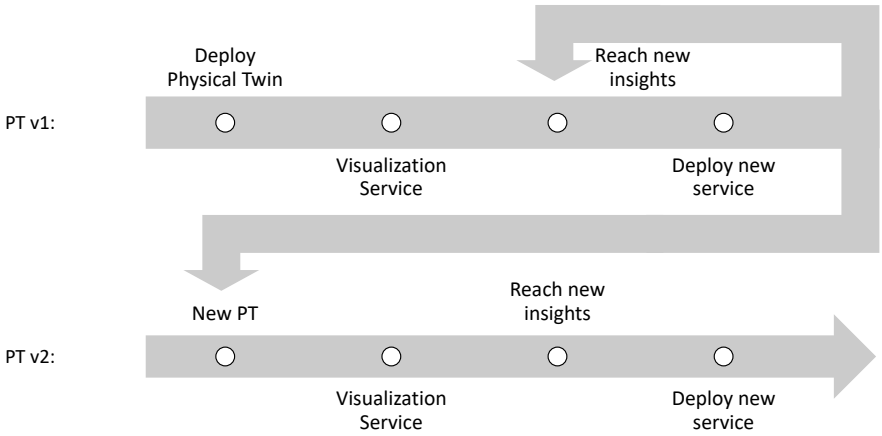

Fig. 12.8: Illustration of the main development steps in the incubator case study, as well as one of the main causes for evolution of the physical twin.

The challenge that arises in the incremental development of DTs is that there are limits to the sensors and actuators of the PT. These in turn set a limit on how advanced the future DT services can be. In the incubator case study, we can highlight this phenomenon by looking at the CO_2 emissions of a healthy tempeh fermentation, displayed in Fig. 12.9. As can be seen, there is significant evidence that such a sensor can bring enormous value to the incubator PT, because it provides an earlier indicator of the temperature of the tempeh rising ahead of healthy fermentation. Note that this sensor was not included in the initial version of the incubator, but it is being included in the second version that is under construction.

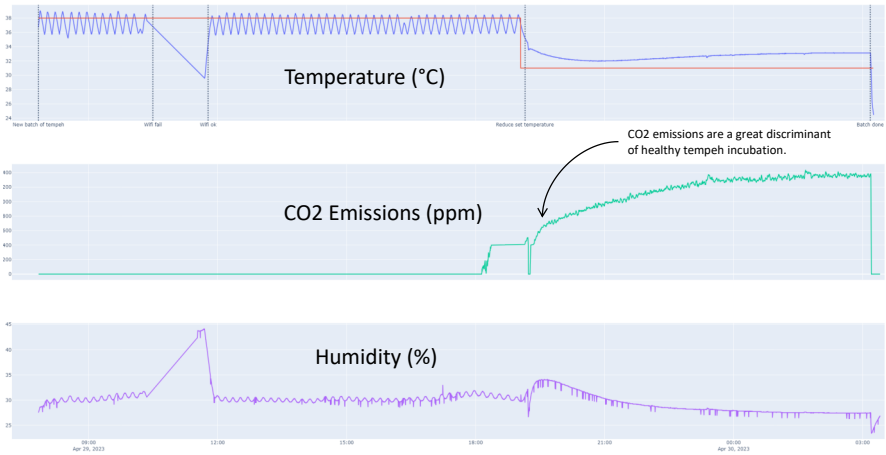

Fig. 12.9: Plot of the data set providing evidence that CO_2 sensor is a great discriminant of healthy tempeh fermentation.

12.3.3 Summary and Future Work

The incubator study provides a small yet representative system for engineering DTs. As the controller must account for the temperature propagation throughout the box, and react to the open lid, the incubator represents some of the challenges seen in twinning CPSs. The source code for the incubator DT is open-source[5], and we invite collaboration on developing further services.

In the short-term, we are enhancing the incubator DT. First, we are further developing visualisations for the incubator based on the Godot game engine. These visualisations will allow the user to control the incubator directly from the visu-

[5] https://digital-twin-book.org/